

Βασικοί (Ενσωματωμένοι) Τύποι Δεδομένων

Τύποι Δεδομένων

Κάθε γλώσσα παρέχει ένα αριθμό **βασικών**
(ενσωματωμένων) τύπων δεδομένων

(π.χ. double, int, char)

και επίσης παρέχει τη δυνατότητα στον
χρήστη να ορίζει **νέους τύπους**

Βασικοί ή βαθμωτοί (scalar) τύποι δεδομένων

- Είναι αυτοί των οποίων οι τιμές **δεν μπορούν να διασπαστούν σε απλούστερα στοιχεία**
- Κάθε γλώσσα προγραμματισμού παρέχει ένα σύνολο βασικών τύπων δεδομένων
 - π.χ. byte, short, int, long, float, double, char και boolean στην Java
- Επιπρόσθετα παρέχει τη δυνατότητα στον προγραμματιστή να ορίζει νέους τύπους
 - Αυτή είναι η μεγάλη δύναμη της Java μέσω των τύπων αναφοράς, δηλαδή των **αντικειμένων**
- Όλα τα δεδομένα αναπαριστούνται στη μνήμη του υπολογιστή ως σειρές δυαδικών ψηφίων (bits)

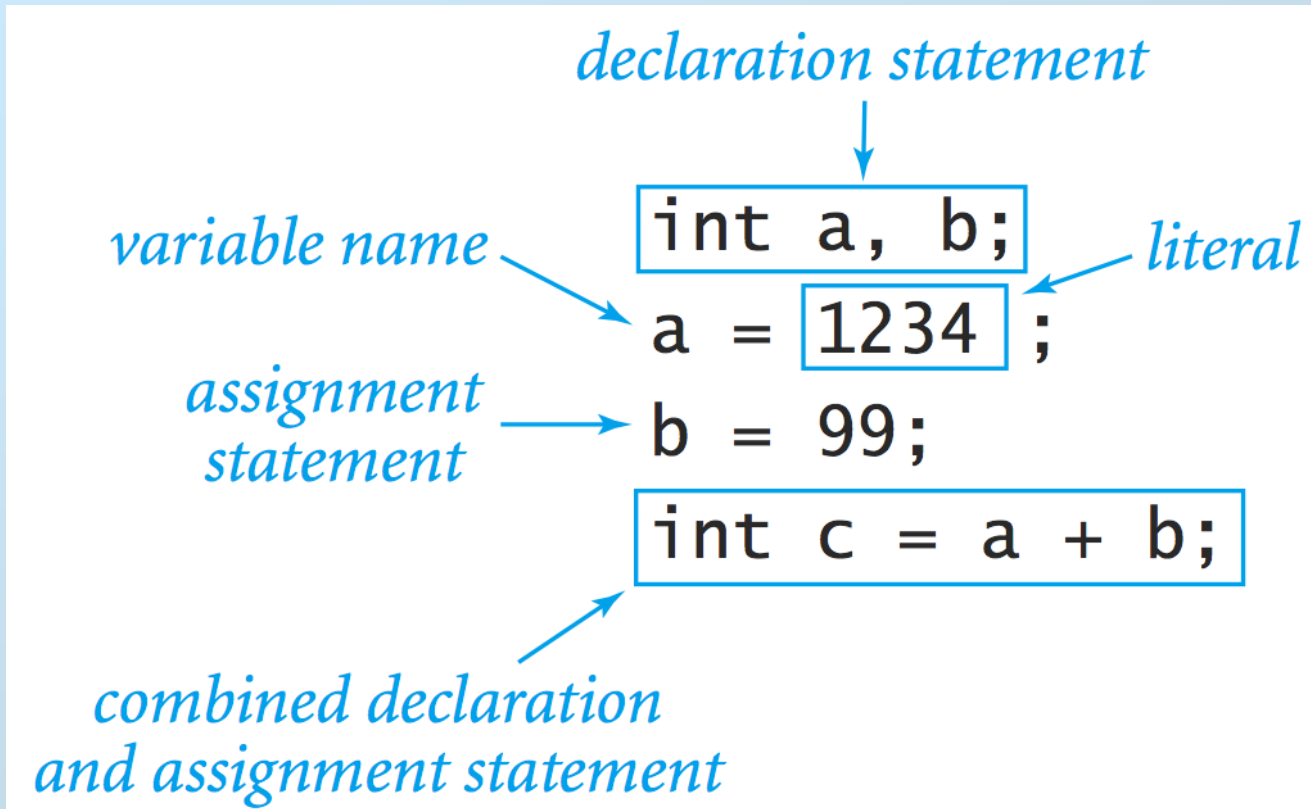
Ενσωματωμένοι Τύποι Δεδομένων

- **Τύπος Δεδομένων** (Data type)
 - Ένα σύνολο **τιμών** (values) και ένα σύνολο **λειτουργιών** (operations) που ορίζονται σε αυτές τις τιμές

ΤΥΠΟΣ	σύνολο τιμών	κυριολεκτικές τιμές	λειτουργίες
char	characters	'A' '@'	compare
String	sequences of characters	"Hello World" "CS is fun"	concatenate
int	integers	17 12345	add, subtract, multiply, divide
double	floating point numbers	3.1415 6.022e23	add, subtract, multiply, divide
boolean	truth values	true false	and, or, not

Βασικοί Ορισμοί

- **Μεταβλητή** (variable)
 - Μια ονομασία που αναφέρεται σε μια τιμή
- **Εντολή ανάθεσης** (assignment statement)
 - Συσχετίζει μια τιμή με μια μεταβλητή



Ίχνος (Trace)

- Πίνακας με τις τιμές των μεταβλητών μετά από κάθε εντολή

	a	b	t
int a, b;	<i>undefined</i>	<i>undefined</i>	
a = 1234;	1234	<i>undefined</i>	
b = 99;	1234	99	
int t = a;	1234	99	1234
a = b;	99	99	1234
b = t;	99	1234	1234

Κείμενο – Τύπος Δεδομένων String (συμβολοσειρά)

- Χρήσιμος για την είσοδο και την έξοδο προγραμμάτων
- Τιμές
 - Ακολουθίες χαρακτήρων
- Κυριολεκτικές τιμές:
 - `"Hello," "1 " " * "`
- Βασική Λειτουργία: concatenate
- Τελεστής: +

Έκφραση	Τιμή
<code>"Hi, " + "Bob"</code>	<code>"Hi, Bob"</code>
<code>"1" + " 2 " + "1"</code>	<code>"1 2 1"</code>
<code>"1234" + " + " + "99"</code>	<code>"1234 + 99"</code>
<code>"1234" + "99"</code>	<code>"123499"</code>

Παράδειγμα: Υποδιαίρέσεις μιας Ρήγας

```
public class Ruler {  
    public static void main(String[] args) {  
        String ruler1 = "1";  
        String ruler2 = ruler1 + " 2 " + ruler1;  
        String ruler3 = ruler2 + " 3 " + ruler2;  
        String ruler4 = ruler3 + " 4 " + ruler3;  
        System.out.println(ruler4);  
    }  
}
```

"1"
"1 2 1"
"1 2 1 3 1 2 1"

String
concatenation

```
% java Ruler  
1 2 1 3 1 2 1 4 1 2 1 3 1 2 1
```



Τύπος Δεδομένων int

- Αναπαριστά **ακεραίους** αριθμούς
- Μέγεθος 4 bytes, πεδίο τιμών: $-2^{31} \dots +2^{31} - 1$
- Παραδείγματα κυριολεκτικών τιμών:
-2189 0 50 +28 -32 1027
- Πράξεις: πρόσθεση (+), αφαίρεση (-),
πολλαπλασιασμός (*), διαίρεση (/), υπόλοιπο (%), κτλ.

<i>expression</i>	<i>value</i>	<i>comment</i>
5 + 3	8	
5 - 3	2	
5 * 3	15	
5 / 3	1	no fractional part
5 % 3	2	remainder
1 / 0		run-time error
3 * 5 - 2	13	* has precedence
3 + 5 / 2	5	/ has precedence
3 - 5 - 2	-4	left associative
(3 - 5) - 2	-4	better style

Παράδειγμα: Πράξεις με Ακεραίους

```
public class IntOps {  
    public static void main(String[] args) {  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
        int sum = a + b;  
        int prod = a * b;  
        int quot = a / b;  
        int rem = a % b;  
        System.out.println(a + " + " + b + " = " + sum);  
        System.out.println(a + " * " + b + " = " + prod);  
        System.out.println(a + " / " + b + " = " + quot);  
        System.out.println(a + " % " + b + " = " + rem);  
    }  
}
```

ορίσματα γραμμής εντολής
(command line arguments)

η Java μετατρέπει
αυτόματα ακεραίους σε
συμβολοσειρές

```

public class IntOps {
    public static void main(String[] args) {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int sum  = a + b;
        int prod = a * b;
        int quot = a / b;
        int rem  = a % b;
        System.out.println(a + " + " + b + " = " + sum);
        System.out.println(a + " * " + b + " = " + prod);
        System.out.println(a + " / " + b + " = " + quot);
        System.out.println(a + " % " + b + " = " + rem);
    }
}

```

```

% javac IntOps.java
% java IntOps 1234 99
1234 + 99 = 1333
1234 * 99 = 122166
1234 / 99 = 12
1234 % 99 = 46

```

1234 = 12*99 + 46

Τύποι δεδομένων για πραγματικούς αριθμούς

- Υπάρχουν δύο τύποι:

<i>Τύπος</i>	<i>Τυπικό Πεδίο Τιμών για Μικροϋπολογιστές</i>	<i>Δεκαδικά Ψηφία</i>
Float (4 bytes)	$10^{-37} \dots 10^{38}$	6
Double (8 bytes)	$10^{-307} \dots 10^{308}$	15

Τύπος Δεδομένων float

(κινήτης υποδιαστολής μονής ακρίβειας)

- Αναπαριστά τους **πραγματικούς** αριθμούς
- Μέγεθος: 4 bytes, 1×10^{37} , 1×10^{-37}
- Πράξεις: πρόσθεση (+), αφαίρεση (-), πολλαπλασιασμός (*), διαίρεση (/), κτλ
- Εκθετική σημειογραφία για κυριολεκτικές τιμές
 - $1258000.0 = 1.258 \times 10^6 = 1.258e6 = 1.258E6$
 - $0.00089 = 8.9 \times 10^{-4} = 8.9e-4$

Ορθές Τιμές

3.14159

0.005

12345.0

15.0e-04 (0.0015)

2.345e2 (234.5)

1.15e-3 (0.00115)

12e+5 (1200000)

Λανθασμένες Τιμές

.12345e

15e-0.3

12.5e.3

34,500.99

Σημείωση: Έννοια της **υπερφόρτωσης** (overloading) – το ίδιο σύμβολο έχει διαφορετικές ερμηνείες σε διαφορετικά συμφραζόμενα, π.χ. τα σύμβολα *, /, -, +, κτλ

Τύπος Δεδομένων **double**

(κινητής υποδιαστολής διπλής ακρίβειας)

- Ίδιος τύπος με float αλλά με μεγαλύτερη ακρίβεια
- Μέγεθος 8 bytes

<i>expression</i>	<i>value</i>
3.141 + .03	3.171
3.141 - .03	3.111
6.02e23 / 2	3.01e23
5.0 / 3.0	1.6666666666666667
10.0 % 3.141	0.577
1.0 / 0.0	Infinity
Math.sqrt(2.0)	1.4142135623730951
Math.sqrt(-1.0)	NaN

Βιβλιοθήκη Math

```
public class Math
```

<code>double abs(double a)</code>	<i>absolute value of a</i>
<code>double max(double a, double b)</code>	<i>maximum of a and b</i>
<code>double min(double a, double b)</code>	<i>minimum of a and b</i>

Note 1: `abs()`, `max()`, and `min()` are defined also for `int`, `long`, and `float`.

<code>double sin(double theta)</code>	<i>sine function</i>
<code>double cos(double theta)</code>	<i>cosine function</i>
<code>double tan(double theta)</code>	<i>tangent function</i>

Note 2: Angles are expressed in radians. Use `toDegrees()` and `toRadians()` to convert.

Note 3: Use `asin()`, `acos()`, and `atan()` for inverse functions.

<code>double exp(double a)</code>	<i>exponential (e^a)</i>
<code>double log(double a)</code>	<i>natural log ($\log_e a$, or $\ln a$)</i>
<code>double pow(double a, double b)</code>	<i>raise a to the bth power (a^b)</i>

<code>long round(double a)</code>	<i>round to the nearest integer</i>
<code>double random()</code>	<i>random number in $[0, 1)$</i>
<code>double sqrt(double a)</code>	<i>square root of a</i>

<code>double E</code>	<i>value of e (constant)</i>
<code>double PI</code>	<i>value of π (constant)</i>

See booksite for other available functions.

Excerpts from Java's mathematics library

Παράδειγμα: Ρίζες Δευτεροβάθμιας Εξίσωσης

$$x^2 + bx + c = 0$$

$$\text{roots} = \frac{-b \pm \sqrt{b^2 - 4c}}{2}$$

Σχέδιο Λύσης:

read from input b and c

$\text{rdiscr} = (b^2 - 4c)^{0.5};$

$x1 = (-b + \text{rdiscr})/2;$

$x2 = (-b - \text{rdiscr})/2;$

print x1 and x2

```
public class Quadratic {  
    public static void main(String[] args) {  
        // parse coefficients from command-line  
        double b = Double.parseDouble(args[0]);  
        double c = Double.parseDouble(args[1]);  
  
        // calculate roots  
        double discriminant = b*b - 4.0*c;  
        double d = Math.sqrt(discriminant);  
        double root1 = (-b + d) / 2.0;  
        double root2 = (-b - d) / 2.0;  
  
        // print them out  
        System.out.println(root1);  
        System.out.println(root2);  
    }  
}
```

Δοκιμή Προγράμματος

- Περιλαμβάνει έγκυρες και μη έγκυρες περιπτώσεις

```
% java Quadratic -3.0 2.0
2.0
1.0
```

← command-line arguments

$$x^2 - 3x + 2$$

```
% java Quadratic -1.0 -1.0
1.618033988749895
-0.6180339887498949
```

← golden ratio

$$x^2 - x - 1$$

```
% java Quadratic 1.0 1.0
NaN
NaN
```

← not a number

$$x^2 + x + 1$$

```
% java Quadratic 1.0 hello
java.lang.NumberFormatException: hello
```

```
% java Quadratic 1.0
java.lang.ArrayIndexOutOfBoundsException
```

Βασικοί Αριθμητικοί Τύποι:

Εσωτερική Αναπαράσταση

Εσωτερικές μορφές τύπων int και double

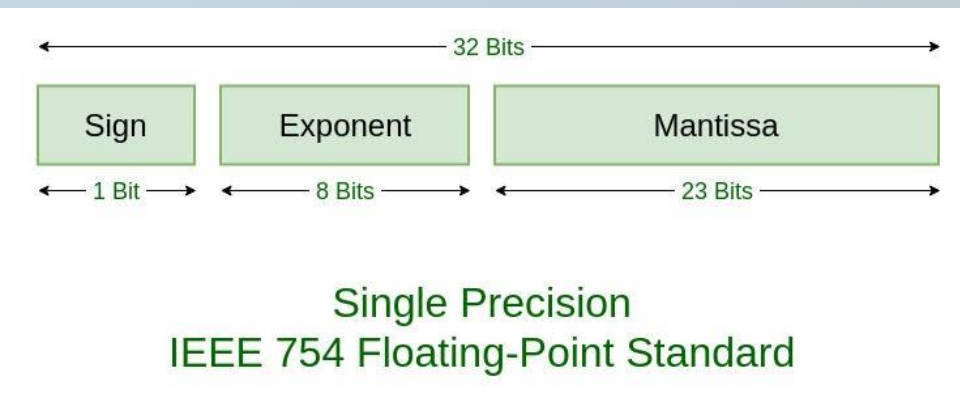
μορφή int

δυναδικός αριθμός

μορφή double

mantissa	εκθέτης
----------	---------

Θετικοί ακέραιοι αριθμοί αναπαριστούνται ως βασικοί δυαδικοί αριθμοί, π.χ. ο ακέραιος αριθμός 13, αναπαριστάται ως 01101



Τύπος double

- Η μορφή του τύπου double, η οποία επίσης ονομάζεται μορφή **σημείου κινητής υποδιαστολής** (floating-point format) είναι ανάλογη της επιστημονικής σημειογραφίας
- Η **mantissa** είναι ένας δυαδικός κλασματικός αριθμός από 0.5 μέχρι 1.0 για θετικούς αριθμούς, και από -0.5 μέχρι -1.0 για αρνητικούς αριθμούς
- Ο **εκθέτης** είναι ένας ακέραιος

$$\text{πραγματικός αριθμός} = \text{mantissa} \times 2^{\text{εκθέτης}}$$

- Επομένως τιμές τύπου double μπορούν να περιέχουν δεκαδικά σημεία, ενώ τιμές τύπου int δεν μπορούν
- Επίσης το πεδίο τιμών του τύπου double είναι πολύ μεγαλύτερο από το πεδίο τιμών του τύπου int
- Τα συγκεκριμένα πεδία τιμών συνήθως διαφέρουν από υλοποίηση σε υλοποίηση της γλώσσας

Άλλοι τύποι δεδομένων για ακέραιους

- Εκτός από τον τύπο `int`, παρέχονται και άλλοι τύποι δεδομένων για ακέραιους αριθμούς:

Τύπος

byte (8 bit)

short (16 bit)

int (32 bit)

long (64 bit).

Τυπικό Πεδίο Τιμών για Μικροϋπολογιστές

– 128 .. 127

– 32,768 .. 32,767

– 2,147,483,648 .. 2,147,483,647

– 9,223,372,036,854,775,808 .. 9,223,372,036,854,775,807

Τύπος δεδομένων `boolean` για λογικές τιμές

- Χρησιμεύει στον έλεγχο της λογικής και της ροής του προγράμματος
- Σύνολο τιμών: `true`, `false`
- Λειτουργίες:
 - Σύζευξη (λογικό και)
 - Διάζευξη (λογικό ή)
 - Άρνηση
- Τελεστές: `&&` `||` `!`

a	!a	a	b	a && b	a b
true	false	false	false	false	false
false	true	false	true	false	true
		true	false	false	true
		true	true	true	true

Truth-table definitions of boolean operations

Συγκριτικοί Τελεστές

- Λαμβάνουν τελεστέους ενός τύπου και δίνουν αποτέλεσμα τύπου `boolean`
- Ισότητα `==`
- Μη ισότητα `!=`
- Μικρότερο `<`
- Μικρότερο ή ίσο `<=`
- Μεγαλύτερο `>`
- Μεγαλύτερο ή ίσο `>=`

<i>op</i>	<i>meaning</i>	<i>true</i>	<i>false</i>
<code>==</code>	<i>equal</i>	<code>2 == 2</code>	<code>2 == 3</code>
<code>!=</code>	<i>not equal</i>	<code>3 != 2</code>	<code>2 != 2</code>
<code><</code>	<i>less than</i>	<code>2 < 13</code>	<code>2 < 2</code>
<code><=</code>	<i>less than or equal</i>	<code>2 <= 2</code>	<code>3 <= 2</code>
<code>></code>	<i>greater than</i>	<code>13 > 2</code>	<code>2 > 13</code>
<code>>=</code>	<i>greater than or equal</i>	<code>3 >= 2</code>	<code>2 >= 3</code>

non-negative discriminant?

`(b*b - 4.0*a*c) >= 0.0`

beginning of a century?

`(year % 100) == 0`

legal month?

`(month >= 1) && (month <= 12)`

Παράδειγμα: Είναι δεδομένο έτος δίσεκτο;

Είναι εάν είτε (i) διαιρείται με το 400, ή
(ii) διαιρείται με το 4 αλλά όχι το 100

Εναλλακτική διατύπωση της συνθήκης:
(i) διαιρείται με το 4, και
(ii) εάν διαιρείται με το 100, τότε πρέπει να
διαιρείται και με το 400

```
// First definition

public class LeapYear {
    public static void main(String[] args) {
        int year = Integer.parseInt(args[0]);
        boolean isLeapYear;

        // divisible by 4 but not 100
        isLeapYear = (year % 4 == 0) && (year % 100 != 0);

        // or divisible by 400
        isLeapYear = isLeapYear || (year % 400 == 0);

        System.out.println(isLeapYear);
    }
}
```

```
% java LeapYear 2004
true
% java LeapYear 1900
false
% java LeapYear 2000
true
```

```
// Second definition

public class LeapYear {
    public static void main(String[] args) {
        int year = Integer.parseInt(args[0]);
        boolean isLeapYear;

        // divisible by 4 and
        // if divisible by 100, it should also be
        // divisible by 400 (not divisible by 100 or
        // divisible by 400)
        isLeapYear = (year % 4 == 0) &&
                     (year % 100 != 0 || year % 400 == 0);

        System.out.println(isLeapYear);
    }
}
```

```
% java LeapYear 2004
true
% java LeapYear 1900
false
% java LeapYear 2000
true
```



```
// Άρθρωση και Επαναχρησιμοποίηση
```

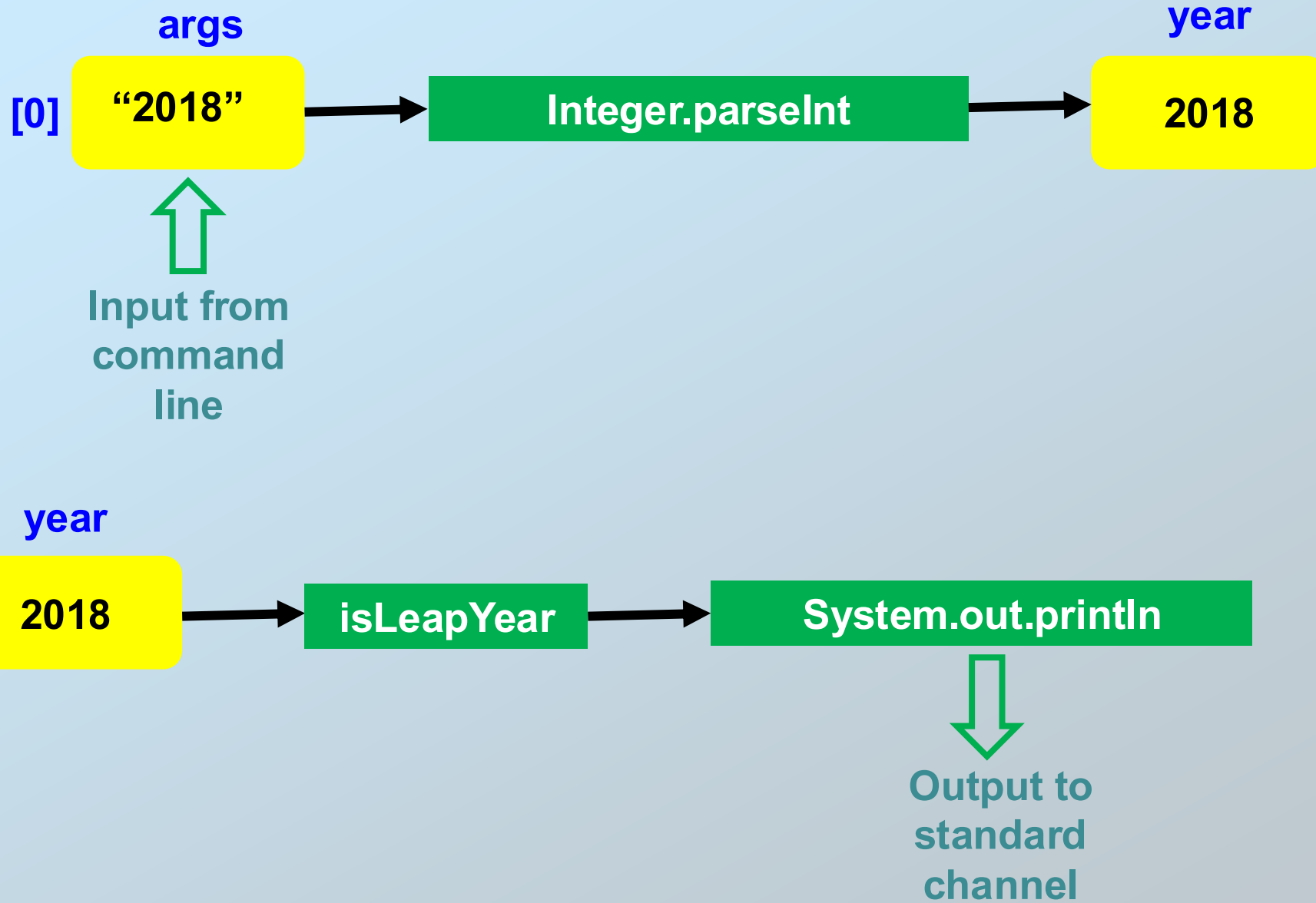
```
public class LeapYear {
```

```
    public static boolean isLeapYear(int year){  
        return (year % 4 == 0) &&  
                (year % 100 != 0 || year % 400 == 0);  
    }
```

```
    public static void main(String[] args) {  
        int year = Integer.parseInt(args[0]);  
        System.out.println(isLeapYear(year));  
    }  
}
```

```
% java LeapYear 2004  
true  
% java LeapYear 1900  
false  
% java LeapYear 2000  
true
```

main



Τύπος Δεδομένων char

- Αναπαριστά ατομικούς χαρακτήρες
 - γράμματα (A-Z, a-z)
 - δεκαδικά ψηφία (0-9)
 - ειδικά σύμβολα (\n, κτλ.)
- Κυριολεκτικές τιμές εσωκλείονται σε αποστρόφους
 - π.χ. 'A', 'a', '9', "'", ' ', '*', '\n', '\'', κτλ.
 - Οι απόστροφος δεν χρειάζονται όταν εισάγονται χαρακτήρες από κάποια μονάδα εισόδου, π.χ. στη γραμμή εντολής

Βαθμωτοί (απαριθμητοί τύποι)

- Οι τύποι `int` και `char` είναι βαθμωτοί (απαριθμητοί) τύποι
- Αναφορικά με τις τιμές τους ορίζονται οι σχέσεις
 - προκάτοχος
 - διάδοχος

Τύπος char

- Ο τελεστής ανάθεσης και οι σχεσιακοί τελεστές μπορούν να εφαρμοστούν σε χαρακτήρες
- Εσωτερικά κάθε χαρακτήρας έχει το δικό του αριθμητικό κωδικό
 - Η τιμή μίας μεταβλητής τύπου char, είναι ένας τέτοιος κωδικός σε δυαδική μορφή
- Υπάρχουν διάφορα **συστήματα κωδικοποίησης χαρακτήρων**:
 - π.χ. το σύστημα ASCII (American Standard Code for Information Interchange)
 - σε κάθε ένα από αυτά τα συστήματα κωδικοποίησης οι χαρακτήρες διατάσσονται σε συγκεκριμένη σειρά. Για παράδειγμα, στο σύστημα ASCII
 - τα αριθμητικά ψηφία, '0' – '9', έχουν κωδικούς 48 – 57
 - τα μικρά γράμματα είναι σε συνεχόμενες θέσεις της διάταξης, καθώς επίσης τα κεφαλαία γράμματα
 - τα κεφαλαία γράμματα προηγούνται των μικρών

Σύγκριση και Εκτύπωση Χαρακτήρων

- Όταν συγκρίνονται χαρακτήρες, το τι συγκρίνονται είναι οι κωδικοί τους
- Στο σύστημα ASCII οι εκτυπώσιμοι χαρακτήρες έχουν κωδικούς από το 32 (' ') μέχρι το 126 ('~'). Οι υπόλοιποι χαρακτήρες είναι **χαρακτήρες ελέγχου** (control characters)
- Η 'εκτύπωση' χαρακτήρων ελέγχου έχει ως αποτέλεσμα η μονάδα εξόδου να εκτελέσει μία ειδική λειτουργία, όπως επιστροφή του δρομέα στη στήλη 1, τοποθέτηση του δρομέα στην επόμενη γραμμή, να ηχήσει κάποιο κουδούνι, κτλ.
- Επειδή εσωτερικά οι χαρακτήρες αναπαριστούνται ως ακέραιοι αριθμοί, η μετατροπή από char σε int και αντιστρόφως επιτρέπεται

```
$ java PrintChars
```

Displaying Print Characters with their Codes

```
 32 ! 33 " 34 # 35 $ 36  
% 37 & 38 ' 39 ( 40 ) 41  
* 42 + 43 , 44 - 45 . 46  
/ 47 0 48 1 49 2 50 3 51  
4 52 5 53 6 54 7 55 8 56  
9 57 : 58 ; 59 < 60 = 61  
> 62 ? 63 @ 64 A 65 B 66  
C 67 D 68 E 69 F 70 G 71  
H 72 I 73 J 74 K 75 L 76  
M 77 N 78 O 79 P 80 Q 81  
R 82 S 83 T 84 U 85 V 86  
W 87 X 88 Y 89 Z 90 [ 91  
\ 92 ] 93 ^ 94 _ 95 ` 96  
a 97 b 98 c 99 d 100 e 101  
f 102 g 103 h 104 i 105 j 106  
k 107 l 108 m 109 n 110 o 111  
p 112 q 113 r 114 s 115 t 116  
u 117 v 118 w 119 x 120 y 121  
z 122 { 123 | 124 } 125 ~ 126
```

Μοναδιαίοι Τελεστές

- Τα σύμβολα $-$ και $+$ (πρόσημο) αντιπροσωπεύουν **μοναδιαίους** τελεστές (unary operators), με μόνο ένα τελεστέο δηλαδή

$$x = -y;$$

$$p = +x * y;$$

- καθώς επίσης **δυναδικούς** τελεστές

$$x = y + z;$$

$$z = y - x;$$

Κανόνες Αποτίμησης Εκφράσεων

- **Κανόνας Παρενθέσεων**

- Κάθε έκφραση σε παρένθεση αποτιμείται ξεχωριστά
- Φωλιασμένες παρενθετικές εκφράσεις, $((..))$, αποτιμούνται από τα μέσα προς τα έξω

- **Κανόνας Προτεραιότητας Τελεστών**

- Τελεστές στην ίδια έκφραση αποτιμώνται σύμφωνα με τις προτεραιότητές τους. Οι προτεραιότητες των αριθμητικών τελεστών, σε φθίνουσα σειρά, είναι

- | | |
|------------------------|--------------------------|
| • μοναδιαίοι $+$, $-$ | υψηλότερη προτεραιότητα |
| • $*$, $/$, $\%$ | ενδιάμεση προτεραιότητα |
| • δυαδικοί $+$, $-$ | χαμηλότερη προτεραιότητα |

Κανόνες Αποτίμησης Εκφράσεων

- **Κανόνας Εταιρικότητας**

- Μοναδιαίοι τελεστές στην ίδια υπο-έκφραση και με την ίδια προτεραιότητα αποτιμούνται από τα δεξιά προς τα αριστερά

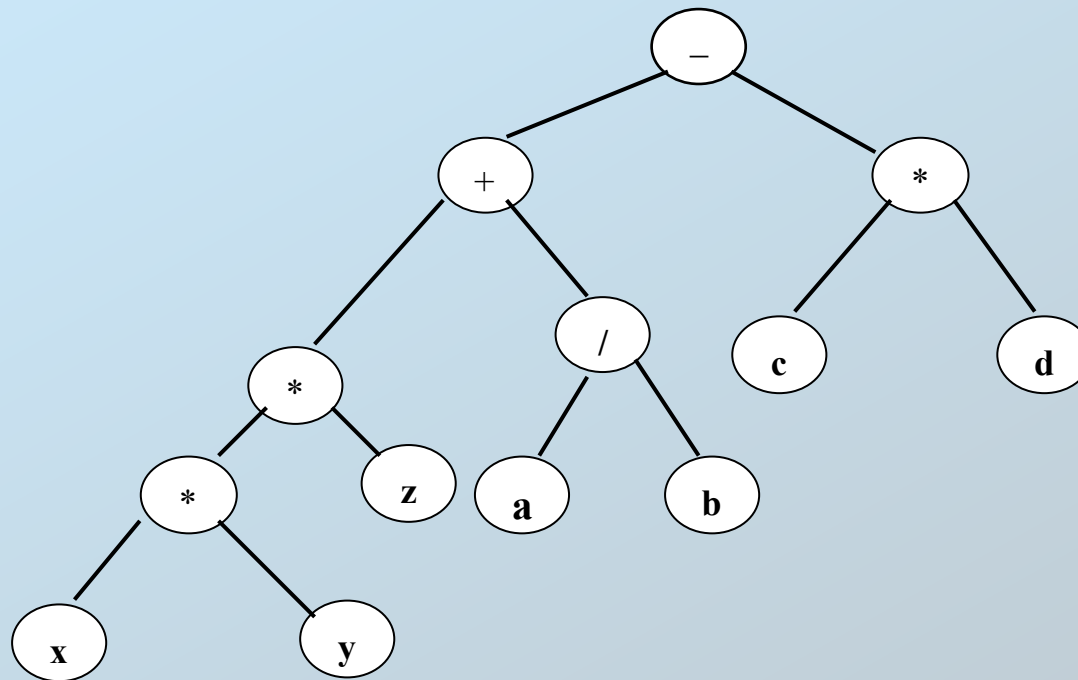
- **δεξιά εταιρικότητα**

- Δυαδικοί τελεστές στην ίδια υπο-έκφραση και με την ίδια προτεραιότητα αποτιμούνται από τα αριστερά προς τα δεξιά

- **αριστερή εταιρικότητα**

Παράδειγμα

Η έκφραση $x * y * z + a / b - c * d$ ερμηνεύεται ως
 $((x * y) * z) + (a / b) - (c * d)$



Όπου υπάρχει αμφιβολία για την σειρά
αποτίμησης, να χρησιμοποιούνται
παρενθέσεις – με μέτρο φυσικά

Συντακτικά Ορθές Εκφράσεις

$x + y / 2$
 $(-x / y) * 2$
 $((3))$
 $+(9 + z)$
 $5 - - x$

Συντακτικά Λανθασμένες Εκφράσεις

$((3 + z) / 2$
 $- 2x / y$
 $5 + * 9$
 $6 ^ 4$

Σημείωση: Δεν υπάρχει τελεστής που υψώνει στη δύναμη. Υπάρχει όμως η μέθοδος `pow` στη Βιβλιοθήκη `Math`

Διατύπωση Εκφράσεων σε Java

- $m^2 - n^2$ $m*m - n*n$ ή
 $(m*m) - (n*n)$
- $ax^2 + bx + c$ $a*x*x + b*x + c$
- $-b + 4ac$ $-b + 4*a*c$
- $\frac{2ab}{c+d}$ $(2*a*b)/(c+d)$
- $((-a)b) + ((-c)d)$ $-a*b + -c*d$

Μετατροπή Τύπων Δεδομένων

- Αυτόματα
 - Χωρίς απώλεια ακρίβειας
 - Με strings
- Ρητά
 - Casting
 - Μέθοδος

Αυτόματη μετατροπή

```
int k = 5, m = 4, n;
```

```
double x = 1.5, y = 2.1, z;
```

```
k + x  $\Rightarrow$  6.5
```

Η τιμή της μεταβλητής k μετατρέπεται σε double πριν την εφαρμογή του τελεστή +

```
z = k / m;
```

Πρώτα αποτιμείται η έκφραση k / m και στη συνέχεια η ακέραια τιμή (1) μετατρέπεται σε double (1.0) προς ανάθεση στη μεταβλητή z

```
n = x * y;
```

Πρώτα αποτιμείται η έκφραση x * y και στη συνέχεια η πραγματική τιμή (3.15) μετατρέπεται σε int (3) προς ανάθεση στη μεταβλητή n

Ρητή μετατροπή

Η ρητή μετατροπή τύπων ονομάζεται **casting**

```
int n1=2, d1=4;  
double frac = (double)n1 / (double)d1;
```

Η έκφραση **(double) 2 / (double) 4** σημαίνει 2.0/4.0, δηλαδή 0.5. Με άλλα λόγια η διαίρεση ακεραίων έχει μετατραπεί σε διαίρεση πραγματικών.

Σε αντίθεση, η έκφραση 2/4 σημαίνει 0. Επομένως η εντολή

```
frac = (double) (n1 / d1);
```

που ισοδυναμεί με την απλούστερη εντολή

```
frac = n1 / d1;
```

όπου γίνεται χρήση της αυτόματης μετατροπής, θα έχει ως αποτέλεσμα την καταχώρηση της τιμής 0.0 στην frac.

Οι εντολές

```
frac = (double) n1 / d1;
```

```
frac = n1 / (double) d1;
```

όπου χρησιμοποιείται συνδυασμός ρητής και αυτόματης μετατροπής ισοδυναμούν με την εντολή

```
frac = (double) n1 / (double) d1;
```


<i>expression</i>	<i>expression type</i>	<i>expression value</i>
<code>"1234" + 99</code>	String	<code>"123499"</code>
<code>Integer.parseInt("123")</code>	int	123
<code>(int) 2.71828</code>	int	2
<code>Math.round(2.71828)</code>	long	3
<code>(int) Math.round(2.71828)</code>	int	3
<code>(int) Math.round(3.14159)</code>	int	3
<code>11 * 0.3</code>	double	3.3
<code>(int) 11 * 0.3</code>	double	3.3
<code>11 * (int) 0.3</code>	int	0
<code>(int) (11 * 0.3)</code>	int	3

Παράδειγμα: Παραγωγή ψευδο-τυχαίου αριθμού ανάμεσα στο 0 και το N-1

```
public class RandomInt {  
    public static void main(String[] args) {  
        int N = Integer.parseInt(args[0]);  
        double r = Math.random();  
        int n = (int) (r * N);  
  
        System.out.println("random integer is " + n);  
    }  
}
```

String to int (method)

double between 0.0 and 1.0 but <1.0

double to int (cast)

int to double (automatic)

int to String (automatic)

```
% java RandomInt 6  
random integer is 3  
% java RandomInt 6  
random integer is 0  
% java RandomInt 10000  
random integer is 3184
```

Ανακρίβειες Αναπαράστασης

- Λάθος αναπαράστασης

- Λάθος το οποίο απορρέει από την κωδικοποίηση ενός πραγματικού αριθμού με χρήση περιορισμένου αριθμού δυαδικών ψηφίων
- Τα λάθη αυτά αναφέρονται και ως **λάθη στρογγυλοποίησης** (round-off errors)
- Για παράδειγμα, ο αριθμός 0.1 έχει λάθος αναπαράστασης ως τιμή τύπου double
- Η επίδραση ενός μικρού λάθους μεγεθύνεται σε επαναληπτικούς υπολογισμούς, με αποτέλεσμα βρόχοι όπως τον ακόλουθο να μην τερματίζουν σε ορισμένους υπολογιστές:

```
for (trial = 0.0; trial != 10.0; trial += 0.1) { ... }
```

Καλύτερα,

```
for (trial = 0.0; trial <= 10.0; trial += 0.1) { ... }
```

Ανακρίβειες Αναπαράστασης

- **Λάθος ακύρωσης**

- λάθος το οποίο απορρέει από την εφαρμογή αριθμητικών τελεστών, σε τελεστέους με πολύ διαφορετικά μεγέθη
- επί της ουσίας ο μικρότερος τελεστέος χάνεται

- **Λάθος υπορροής (underflow error)**

- συμβαίνει όταν μία πολύ μικρή αριθμητική τιμή αναπαριστάται ως το μηδέν

- **Λάθος υπερχείλισης (overflow error)**

- συμβαίνει όταν η αριθμητική τιμή που χρειάζεται να αναπαρασταθεί είναι πολύ μεγάλη
- αυτό μπορεί να συμβεί και με ακεραίους αριθμούς

```
public class Main{  
    public static void main(String[] args) {  
        double a = 1;  
        double b = a + 0.000000000000000001;  
        System.out.print(a + " " + b);  
    }  
}
```

1.0 1.0

```
public class Main{  
    public static void main(String[] args) {  
        double a = 0.01;  
        double b = a + 0.00000000000000000001;  
        System.out.print(a + " " + b);  
    }  
}
```

0.01 0.01

```
public class Main{  
    public static void main(String[] args) {  
        int a = 2147483647;  
        int b = a + 1;  
        System.out.print(a + " " + b);  
    }  
}
```

2147483647 -2147483648

Σύνοψη

- Ένας τύπος δεδομένων είναι ένα σύνολο τιμών και λειτουργιών στις τιμές αυτές
 - `String` επεξεργασία κειμένου
 - `double, int` μαθηματικοί υπολογισμοί
 - `boolean` αποφάσεις
- Προσοχή
 - Οι τύποι των τιμών δηλώνονται
 - Μετατρέπουμε ανάμεσα σε τύπους όπου είναι αναγκαίο
 - Το 1996, ο πύραυλος Ariane 5 rocket εξερράγη μετά την απογείωση λόγω κακής μετατροπής τύπου

